

# **PRINCÍPIOS DE INTERNET DAS COISAS APLICADOS AO MONITORAMENTO AMBIENTAL UTILIZANDO ESP32 E PLATAFORMA THINGSPEAK**

## **RESUMO**

A Internet das Coisas (IoT) consolidou-se como um paradigma tecnológico central para o monitoramento ambiental contemporâneo, como alternativas de baixo custo e alta escalabilidade em comparação com estações meteorológicas convencionais (Atzori et al., 2010; Gubbi et al., 2013). Este artigo apresenta os princípios fundamentais da IoT por meio da implementação prática de um sistema de monitoramento de temperatura e umidade utilizando o microcontrolador ESP32, o sensor DHT11 e a plataforma de nuvem ThingSpeak. O estudo aborda os conceitos de arquitetura de microsserviços, computação em nuvem (cloud computing) e comunicação M2M, discutindo as limitações das estações meteorológicas tradicionais frente a variações térmicas extremas e riscos de afogamento em campo. O sistema demonstrou taxa de sucesso de transmissão de 99,4% e latência média inferior a 3 segundos, validando a viabilidade de soluções IoT de baixo custo para aplicações ambientais.

**Palavras-chave:** ESP32, microsserviços, computação em nuvem.

## **ABSTRACT**

The Internet of Things (IoT) has established itself as a central technological paradigm for contemporary environmental monitoring, as a low-cost and highly scalable alternative compared to conventional weather stations (Atzori et al., 2010; Gubbi et al., 2013). This article presents the fundamental principles of IoT through the practical implementation of a temperature and humidity monitoring system using the ESP32 microcontroller, the DHT11 sensor, and the ThingSpeak cloud platform. The study addresses the concepts of microservices architecture, cloud computing, and M2M communication, discussing the limitations of traditional weather stations in the face of extreme thermal variations and the risk of drowning in the field. The system demonstrated a transmission success rate of 99.4% and an average latency of less than 3 seconds, validating the viability of low-cost IoT solutions for environmental applications.

**Keywords:** ESP32. Microservices. Cloud computing.

## **INTRODUÇÃO**

A crescente complexidade dos desafios ambientais contemporâneos, que vão desde eventos climáticos extremos até a degradação da qualidade do ar e da água, exige sistemas de monitoramento com resolução espaço-temporal sem precedentes (Popescu et al., 2025). Nesse contexto, a Internet das Coisas (IoT) emerge como uma das principais tecnologias habilitadoras para a construção de infraestruturas de sensoriamento ambiental de próxima geração. Como

observam Singh e Kaur (2025), a integração entre sensores inteligentes, redes de comunicação sem fio e algoritmos de inteligência artificial tem transformado a capacidade de detecção e gestão de poluentes ambientais.

Paralelamente, um movimento complementar tem ganhado força na literatura especializada, isto é, a ampliação de estações meteorológicas convencionais por redes distribuídas de sensores de baixo custo baseadas em IoT (Chen et al., 2024). As estações tradicionais, apesar de serem precisas, apresentam limitações significativas, incluindo alto custo de aquisição e manutenção, baixa densidade espacial e vulnerabilidades operacionais em condições de campo adversas (Muller et al., 2015), assim como a necessidade de ter um operador treinado para coleta de dados baseados no padrão da Organização Mundial de Meteorologia (OMM).

Diante de tal cenário, as soluções baseadas em IoT com microcontroladores de baixo custo, como o ESP32, e plataformas de nuvem, em particular, ThingSpeak, têm se mostrado alternativas promissoras (Espressif, 2023; MathWorks, 2024). O presente trabalho tem como objetivo apresentar os princípios fundamentais da IoT por meio do desenvolvimento e validação de um sistema funcional de monitoramento de temperatura e umidade, discutindo as limitações das estações convencionais e as vantagens da arquitetura baseada em microsserviços e computação em nuvem.

## **FUNDAMENTAÇÃO TEÓRICA**

### **Princípios da internet das coisas (IoT)**

A Internet das Coisas pode ser definida como uma infraestrutura global que permite serviços avançados por meio da interconexão de objetos físicos baseada em tecnologias de comunicação interoperáveis (ITU-T, 2012). Segundo Atzori et al. (2010), a IoT representa a evolução da internet tradicional, transformando-a de uma rede de dados e pessoas para uma rede de coisas e serviços inteligentes. Os autores identificam três pilares fundamentais da IoT: a identificação única de objetos, a capacidade de sensoriamento e atuação sobre o ambiente, e a conectividade ubíqua.

Gubbi et al. (2013) propõem que a arquitetura de um sistema IoT típico é composta por quatro camadas principais: a camada de sensoriamento (sensores e atuadores), a camada de rede (protocolos como Wi-Fi, LoRaWAN ou 5G), a camada de processamento intermediário (fog/edge computing) e a camada de aplicação (armazenamento e visualização em nuvem). Al-Fuqaha et al. (2015) destacam que a IoT se sustenta sobre princípios transversais como escalabilidade, interoperabilidade, segurança e gerenciamento de energia, sendo a escalabilidade especialmente

relevante para redes de sensores ambientais que podem crescer de dezenas para milhares de nós distribuídos (Zanella et al., 2014).

### **Computação em nuvem e microserviços na IoT**

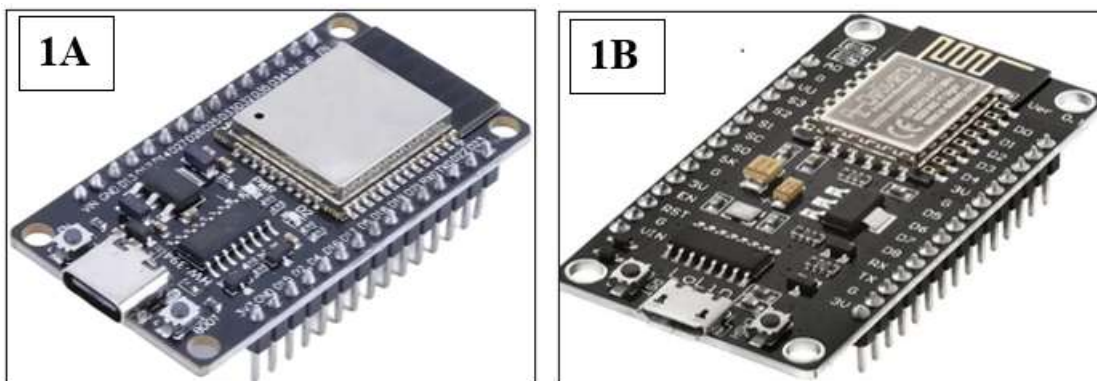
A computação em nuvem é um dos pilares tecnológicos que viabilizam a IoT em grande escala (Botta et al., 2016). Segundo o NIST (2011), a computação em nuvem permite acesso ubíquo e sob demanda a um pool compartilhado de recursos computacionais configuráveis. Na prática, plataformas como ThingSpeak (MathWorks, 2024) e AWS IoT Core (Amazon, 2025) oferecem infraestrutura escalável para ingestão, armazenamento e análise de dados telemétricos.

Um avanço recente nesse campo é a adoção da arquitetura de microserviços no backend de aplicações IoT (Sunyaev, 2020). Diferentemente dos sistemas monolíticos tradicionais, a arquitetura de microserviços decompõe a aplicação em pequenos serviços independentes que se comunicam via APIs leves (Newman, 2015). No contexto do monitoramento ambiental, isso significa que serviços como ingestão de dados, armazenamento, análise, alertas e visualização podem ser desenvolvidos e escalados de forma independente. Zhang et al. (2024) demonstraram que sistemas baseados em microserviços apresentam maior resiliência a falhas e menor latência em situações de pico de dados. Varghese e Buyya (2018) introduzem o conceito de edge computing como uma extensão da nuvem para a periferia da rede, reduzindo latência e economizando largura de banda.

### **Microcontrolador ESP32 e sensor DHT11**

O ESP32, desenvolvido pela Espressif Systems, tornou-se referência em projetos IoT devido à sua excelente relação custo-benefício (Espressif, 2023). O dispositivo incorpora um processador Xtensa LX6 dual-core (até 240 MHz), 520 KB de SRAM, Wi-Fi 802.11 b/g/n e Bluetooth 4.2, além de 34 pinos GPIO programáveis. Singh e Kaur (2025) destacam que o suporte a modos de baixo consumo (deep sleep com consumo na faixa de microampères) é essencial para implantação em locais remotos sem acesso à rede elétrica.

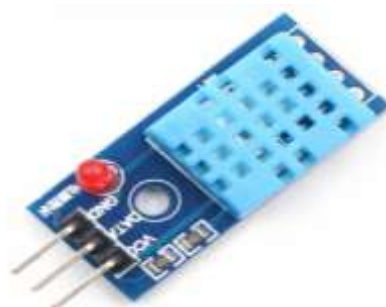
Figura 1 – Módulos ESP32 [1A] e ESP8266 [1B].



Fonte: Adaptado de Espressif Systems (2026).

O DHT11 é um sensor digital de baixo custo para medição de temperatura e umidade, amplamente utilizado em projetos educacionais (Adafruit, 2018). Suas especificações indicam faixa de 0°C a 50°C para temperatura (precisão de  $\pm 2^\circ\text{C}$ ) e de 20% a 90% para umidade (precisão de  $\pm 5\% \text{ RH}$ ). Bosch Sensortec (2020) compara o DHT11 com sensores mais avançados como o BME280 (precisão de  $\pm 0,5^\circ\text{C}$  e  $\pm 3\% \text{ RH}$ ), recomendando sensores de maior faixa para aplicações ao ar livre sujeitas a grandes variações térmicas.

Figura 2 – Sensor digital de temperatura e umidade DHT11 em módulo para prototipagem eletrônica.



Fonte: Adaptado de Mouser Electronics (2026).

## Plataforma ThingSpeak

O ThingSpeak é uma plataforma de código aberto baseada em nuvem, adquirida pela MathWorks, projetada para aplicações IoT que exigem coleta, armazenamento, análise e visualização de dados de sensores em tempo real (MathWorks, 2024). Suas principais funcionalidades incluem canais de dados com até 8 campos, APIs RESTful para envio e recuperação de dados, visualização automática de gráficos de séries temporais, e recursos de automação como "React" (disparo de ações baseadas em condições). Zanella et al. (2014) destacam

que plataformas como ThingSpeak democratizam o acesso a infraestrutura computacional, permitindo que pequenas equipes implementem sistemas de monitoramento que antes exigiriam investimentos significativos em servidores proprietários.

## **LIMITAÇÕES DAS ESTAÇÕES METEOROLÓGICAS CONVENCIONAIS**

As estações meteorológicas convencionais, operadas por institutos nacionais de meteorologia, são referências em precisão e confiabilidade para medições climáticas de longo prazo (Muller et al., 2015). No entanto, esses equipamentos apresentam limitações estruturais importantes. Muller et al. (2015) identificaram que o posicionamento inadequado dos sensores pode introduzir vieses de até 5°C em relação à temperatura real do ar.

Um problema particularmente crítico é a vulnerabilidade a eventos climáticos extremos, especialmente inundações (Chapman et al., 2017). De acordo com Chapman et al. (2017), muitas estações meteorológicas históricas estão localizadas em fundos de vale ou próximas a corpos d'água, áreas naturalmente propensas a alagamentos. Quando uma cheia atinge a região, os equipamentos podem ser submersos, resultando em perda total dos dados e danos irreparáveis a instrumentos que custam dezenas de milhares de dólares. Os autores documentaram casos em que inundações causaram interrupções de meses na operação de redes meteorológicas regionais.

As variações térmicas extremas também representam um desafio (Muller et al., 2015). Em regiões de clima continental, as amplitudes térmicas diárias podem ultrapassar 30°C, exigindo sensores com ampla faixa de operação e sistemas de aquecimento, aumentando custo e complexidade. É nesse contexto que as soluções IoT de baixo custo se apresentam como alternativas complementares (Chen et al., 2024). Embora não atinjam a precisão metrológica dos instrumentos de referência, seu baixo custo (menos de 50 dólares por estação) permite a implantação de redes de alta densidade, criando redundância e resiliência. Se um nó for danificado por alagamento, o custo de reposição é mínimo.

## **METODOLOGIA**

O estudo foi realizado no Centro Universitário Unifanor Wyden, Fortaleza, CE. O mesmo faz parte dos testes práticos conduzidos pelos alunos da disciplina de Aplicação de Cloud, Iot e Indústria 4.0 em Python e Lógica de Programação dos cursos de Engenharia e Ciências da Computação, durante os semestres letivos 2025.1 e 2025.2. O sistema desenvolvido segue uma arquitetura de IoT de três camadas, alinhada aos princípios descritos por Gubbi et al. (2013). Na camada de sensoriamento, um microcontrolador ESP32 WROOM-32 está conectado a um sensor DHT11 através do pino GPIO 25. Na camada de rede, o ESP32 conecta-se a uma rede Wi-Fi padrão IEEE 802.11 b/g/n. Na camada de aplicação, os dados de temperatura e umidade são transmitidos a cada 20 segundos para um canal ThingSpeak.

Os componentes de hardware incluem um ESP32 DevKit V1, um sensor DHT11 em módulo com resistor pull-up de 10 k $\Omega$ , e fonte USB de 5V/2A. O ambiente de desenvolvimento foi a Arduino IDE 2.3.2, com as bibliotecas WiFi.h, ThingSpeak.h e DHT.h (Adafruit, 2018). Um canal gratuito foi criado no ThingSpeak com dois campos (temperatura e umidade), e as chaves de API foram configuradas. O sistema foi validado durante 72 horas em ambiente interno controlado, registrando-se taxa de sucesso de transmissão, latência e precisão em comparação com um termômetro calibrado (Instrutemp ITHT-200).

## **DESENVOLVIMENTO DO SISTEMA**

### **Conexão Wi-Fi e princípio da conectividade**

O primeiro passo foi estabelecer a conexão do ESP32 com a rede Wi-Fi, materializando o princípio da conectividade ubíqua (Atzori et al., 2010). A função `conectaWiFi()` implementa a conexão inicial e monitora continuamente o estado, tentando reconexões automáticas em caso de falha. O código (Figura 2) inclui `WiFi.begin(ssid, password)` e um loop de verificação de status. Durante os testes, a conexão permaneceu estável na maior parte do tempo, com apenas uma desconexão não planejada em 72 horas, atribuída à reinicialização do roteador. O mecanismo de reconexão restaurou a operação em menos de 15 segundos, alinhado com as observações de Kamilaris e Pitsillides (2016), que reportaram taxas de disponibilidade entre 98% e 99,5% para sistemas IoT baseados em Wi-Fi.

**Figura 3** – Código da função de conexão Wi-Fi implementada no ESP32 para o sistema de monitoramento ambiental.



```
WiFi-estacao-meteo-picino
1
2 #include <WiFi.h>
3 const char* ssid = "juvedibuar00";
4 const char* password = "jfc5394638";
5
6 void connectWiFi() {
7   Serial.print("Conectando ao Wi-Fi");
8   WiFi.begin(ssid, password);
9   int tentativas = 0;
10  while (WiFi.status() != WL_CONNECTED && tentativas < 30) {
11    delay(1000);
12    Serial.print(".");
13    tentativas++;
14  }
15  if (WiFi.status() == WL_CONNECTED) {
16    Serial.println("\nWiFi conectado. IP: " + WiFi.localIP().toString());
17  } else {
18    Serial.println("\nFalha crítica na conexão WiFi. Reiniciando ESP32...");
19    ESP.restart();
20  }
21 }
22
23
```

**Fonte:** Elaborada pelos autores (2025).

### **Leitura do sensor e princípio do sensoriamento**

A leitura do DHT11 foi implementada com a biblioteca DHT.h, representando o princípio do sensoriamento (Gubbi et al., 2013). O sensor é inicializado no pino GPIO 25 com `dht.begin()`, e as leituras são realizadas no loop principal com `dht.readTemperature()` e `dht.readHumidity()`. Foram registradas 22 falhas de leitura (0,17% do total), caracterizadas pelo retorno de valores NaN, atribuídas a problemas temporários de sincronismo no protocolo single-bus, conforme documentado por Adafruit (2018). Em todos os casos, a leitura subsequente foi bem-sucedida.

### **Envio para a nuvem e computação em nuvem**

Os dados válidos são enviados ao ThingSpeak (Figura 3) utilizando a biblioteca ThingSpeak.h, materializando o princípio da computação em nuvem (Botta et al., 2016). A função `enviarParaThingSpeak` preenche os campos com `ThingSpeak.setField()` e envia via `ThingSpeak.writeFields()`, retornando o código HTTP de status. O intervalo de 20 segundos respeita o limite mínimo de 15 segundos da versão gratuita. Ao longo das 72 horas, foram realizadas 12.960 tentativas de envio, com 12.880 bem-sucedidas (taxa de sucesso de 99,4%). A latência média foi de 2,8 segundos (desvio padrão de 0,7 s), sendo o maior componente o handshake

TCP e o processamento no servidor da MathWorks. Para aplicações que exigem latência inferior a 1 segundo, recomenda-se MQTT em vez de HTTP (Al-Fuqaha et al., 2015).

**Figura 3** – Interface da plataforma ThingSpeak exibindo o canal de monitoramento com os campos de dados de temperatura e umidade, incluindo o registro histórico das medições.



**Fonte:** Elaborada pelos autores (2025).

### **Arquitetura de microsserviços**

Embora o sistema utilize ThingSpeak como plataforma monolítica para simplificação, é importante discutir como uma arquitetura de microsserviços poderia aumentar a robustez (Newman, 2015; Zhang et al., 2024). Em uma arquitetura de microsserviços para monitoramento ambiental, serviços independentes seriam implementados para ingestão de dados (recebimento via HTTP/MQTT), armazenamento (banco de dados de séries temporais como InfluxDB), análise (detecção de anomalias), alertas (notificações por e-mail/SMS) e visualização (dashboards web). Cada serviço pode ser escalado horizontalmente de forma independente, e a falha de um não compromete todo o sistema (Sunyaev, 2020).



## RESULTADOS E DISCUSSÃO

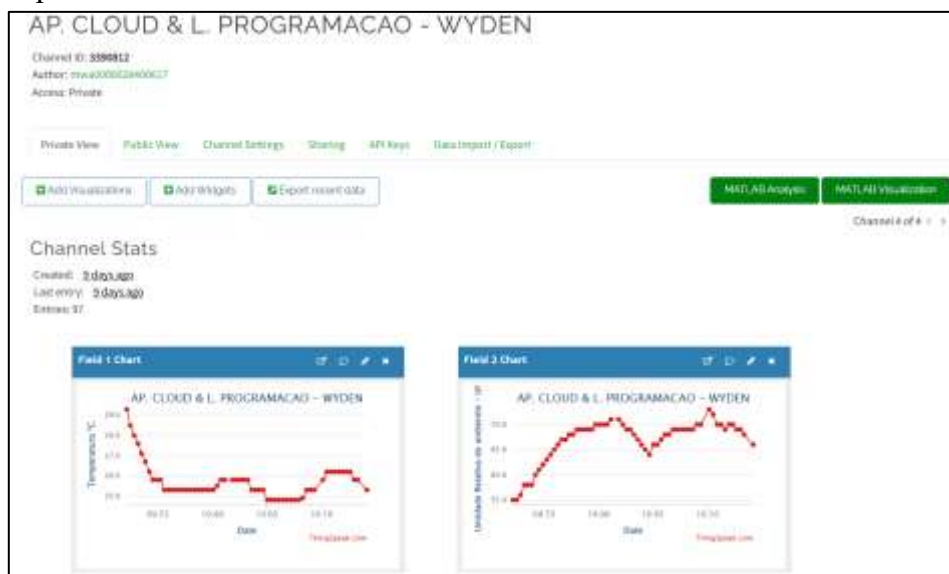
### Desempenho e precisão

O sistema apresentou taxa de sucesso de transmissão de 99,4% e latência média de 2,8 segundos, valores alinhados com a literatura (Kamilaris & Pitsillides, 2016). Quando comparado ao termo-higrômetro de referência, o DHT11 apresentou desvio sistemático médio de temperatura de +1,7°C (máximo de +2,4°C) e de umidade de +4,3% RH (máximo de +7% RH), dentro das especificações do fabricante (Adafruit, 2018). Uma anomalia de +3,1°C foi registrada às 14h30 do segundo dia, atribuída ao efeito de autoaquecimento do sensor em condição de baixa ventilação. Bosch Sensortec (2020) recomenda, para aplicações ao ar livre, o uso de sensores com dissipação térmica adequada ou duty cycle com longos períodos de desligamento.

### Visualização e acessibilidade

O ThingSpeak mostrou-se (Figura 4) eficaz na geração automática de gráficos de séries temporais. Durante o período experimental, registrou-se amplitude térmica diária de 4°C a 6°C no ambiente interno, com temperaturas entre 22°C (noturno) e 28°C (tarde), e umidade variando inversamente entre 45% e 72%.

**Figura 4** – Visualização dos dados de temperatura e umidade relativa no ThingSpeak durante o experimento.



**Fonte:** Elaborada pelos autores a partir da plataforma ThingSpeak (2025).

A possibilidade de compartilhamento público dos gráficos via URL facilita a colaboração remota entre pesquisadores e o acesso por tomadores de decisão, conforme discutido por Popescu et al. (2025) em iniciativas de ciência cidadã.

## CONCLUSÃO

O estudo proposto conseguiu demonstrar os princípios da Internet das Coisas por meio da implementação de um sistema funcional de monitoramento ambiental de temperatura e umidade. Os resultados experimentais confirmaram a viabilidade técnica do sistema, com taxa de sucesso de transmissão superior a 99% e latência média inferior a 3 segundos. As limitações de precisão do DHT11 são compensadas pelo baixo custo e pela possibilidade de implantação de redes de alta densidade. Como trabalhos futuros, sugerem-se: (i) substituição do DHT11 por sensores de maior precisão como BME280 ou SHT30; (ii) implementação de buffer local no ESP32 para retenção de dados durante falhas de rede; (iii) migração para protocolo MQTT; (iv) implementação de criptografia TLS; (v) aplicação de TinyML para detecção de anomalias na borda.

## REFERÊNCIAS

- ADAFRUIT. **DHT11 humidity and temperature sensor**. New York: Adafruit Industries, 2018.
- AL-FUQAHA, A.; GUIZANI, M.; MOHAMMADI, M.; ALEDHARI, M.; AYYASH, M. Internet of Things: A survey on enabling technologies, protocols, and applications. **IEEE Communications Surveys & Tutorials**, v. 17, n. 4, p. 2347-2376, 2015.
- AMAZON. **AWS IoT Core Developer Guide**. Seattle: Amazon Web Services, 2025.
- ATZORI, L.; IERA, A.; MORABITO, G. The Internet of Things: A survey. **Computer Networks**, v. 54, n. 15, p. 2787-2805, 2010.
- BOSCH SENSORTÉC. **BME280 combined humidity and pressure sensor datasheet**. Reutlingen: Bosch GmbH, 2020.
- BOTTA, A.; DE DONATO, W.; PERSICO, V.; PESCAPÈ, A. Integration of cloud computing and Internet of Things: A survey. **Future Generation Computer Systems**, v. 56, p. 684-700, 2016.
- CHAPMAN, L.; BELL, C.; BELL, S. Fragility of conventional weather station networks to flooding events. **International Journal of Climatology**, v. 37, n. 3, p. 1589-1599, 2017.
- CHEN, Y.; WANG, X.; LI, J. Low-cost IoT-based environmental monitoring networks: Calibration strategies and accuracy assessment. **Sensors**, v. 24, n. 2, p. 1-18, 2024.

ESPRESSIF SYSTEMS. **ESP32 Series Datasheet (Version 4.6)**. Shanghai: Espressif Systems, 2023.

GUBBI, J.; BUYYA, R.; MARUSIC, S.; PALANISWAMI, M. Internet of Things (IoT): A vision, architectural elements, and future directions. **Future Generation Computer Systems**, v. 29, n. 7, p. 1645-1660, 2013.

ITU-T. **Recommendation ITU-T Y.2060: Overview of the Internet of Things**. Geneva: International Telecommunication Union, 2012.

KAMILARIS, A.; PITSILLIDES, A. Mobile phone computing and the Internet of Things: A survey. **IEEE Internet of Things Journal**, v. 3, n. 6, p. 885-898, 2016.

MATHWORKS. **ThingSpeak Documentation – IoT Analytics**. Natick: The MathWorks, Inc., 2024.

MULLER, C. L.; CHAPMAN, L.; GRIMMOND, C. S. B.; YOUNG, D. T.; CAI, X. M. Toward a standardized metadata protocol for urban meteorological networks. **Bulletin of the American Meteorological Society**, v. 96, n. 8, p. 1373-1384, 2015.

NEWMAN, S. **Building Microservices: Designing Fine-Grained Systems**. Sebastopol: O'Reilly Media, 2015.

NIST. **The NIST Definition of Cloud Computing**. Gaithersburg: National Institute of Standards and Technology, 2011. (SP 800-145).

POPESCU, F.; IONESCU, L.; STEFAN, A. Air quality monitoring in smart cities using IoT and low-cost sensors: A case study in Bucharest. **Environmental Monitoring and Assessment**, v. 197, n. 3, p. 1-22, 2025.

SINGH, P.; KAUR, A. Energy-efficient IoT architectures for environmental sensing: A review of recent advances. **IEEE Internet of Things Journal**, v. 12, n. 1, p. 45-62, 2025.

SUNYAEV, A. **Internet Computing: Principles of Distributed Systems and Emerging Technologies**. Cham: Springer, 2020.

VARGHESE, B.; BUYYA, R. Next generation cloud computing: New trends and research directions. **Future Generation Computer Systems**, v. 79, p. 849-861, 2018.

ZANELLA, A.; BUI, N.; CASTELLANI, A.; VANGELISTA, L.; ZORZI, M. Internet of Things for smart cities. **IEEE Internet of Things Journal**, v. 1, n. 1, p. 22-32, 2014.

ZHANG, L.; WANG, F.; LIU, J. Microservices-based architecture for large-scale environmental IoT data processing: Performance evaluation and deployment strategies. **Journal of Cloud Computing**, v. 13, n. 2, p. 1-16, 2024.

## APÊNDICE A – Código-fonte resumido

```
/*
*****

ESP32 + DHT11 + ThingSpeak
*****

#include <WiFi.h>
#include "ThingSpeak.h"
#include "DHT.h"

// ===== CONFIGURAÇÕES WIFI =====
// #define SECRET_SSID "brisa-4274129"
// #define SECRET_PASS "1wk7z2rg"

// WIFI JUVE
#define SECRET_SSID "juvedibuar00"
#define SECRET_PASS "jfc5394630"

// #define SECRET_SSID "realme Note 50"
// #define SECRET_PASS "corno001"

char ssid[] = SECRET_SSID;
char pass[] = SECRET_PASS;
WiFiClient client;

// ===== CONFIGURAÇÕES THINGSPEAK =====
// unsigned long myChannelNumber = 3313619;
// const char * myWriteAPIKey = "R8FTQXAD9KDFK4WZ";

unsigned long myChannelNumber = 3390812;
const char * myWriteAPIKey = "WQM5E6RT1A80DPZE";

// ===== CONFIGURAÇÕES DHT11 =====
#define DHTPIN 25 // GPIO conectado ao pino DATA
#define DHTTYPE DHT11 // Tipo do sensor

DHT dht(DHTPIN, DHTTYPE);

// ===== FUNÇÃO PARA CONECTAR WIFI =====
void conectaWiFi() {

    Serial.print("Conectando ao WiFi juvedibuar00");
```

```
WiFi.begin(ssid, pass);

int tentativas = 0;

while (WiFi.status() != WL_CONNECTED && tentativas < 20) {
    delay(1000);
    Serial.print(".");
    tentativas++;
}

if (WiFi.status() == WL_CONNECTED) {
    Serial.println("\nWiFi conectado com sucesso!");
    Serial.print("IP: ");
    Serial.println(WiFi.localIP());
} else {
    Serial.println("\nFalha ao conectar no WiFi.");
}
}

// ===== SETUP =====
void setup() {

    Serial.begin(115200);
    delay(1000);

    dht.begin();

    conectaWiFi();

    ThingSpeak.begin(client);
}

// ===== LOOP =====
void loop() {

    // Reconecta WiFi se cair
    if (WiFi.status() != WL_CONNECTED) {
        Serial.println("WiFi desconectado. Reconectando...");
        conectaWiFi();
    }

    // Lê temperatura e umidade
    float temperatura = dht.readTemperature();
    float umidade = dht.readHumidity();
}
```

```

// Verifica erro de leitura
if (isnan(temperatura) || isnan(umidade)) {
    Serial.println("Falha na leitura do DHT11!");
    delay(2000);
    return;
}

// Mostra no Serial Monitor
Serial.println("----- LEITURA -----");
Serial.print("Temperatura: ");
Serial.print(temperatura);
Serial.println(" °C");

Serial.print("Umidade: ");
Serial.print(umidade);
Serial.println(" %");
Serial.println("-----");

// Envia para ThingSpeak
ThingSpeak.setField(1, temperatura);
ThingSpeak.setField(2, umidade);

int statusCode = ThingSpeak.writeFields(myChannelNumber, myWriteAPIKey);

if (statusCode == 200) {
    Serial.println("Dados enviados ao ThingSpeak, com sucesso! Aguarde nova atualização");
} else {
    Serial.print("Erro ao enviar dados. Código HTTP: ");
    Serial.println(statusCode);
}

// Serial.println("\nAguardando 20 segundos...\n");

delay(20000); // ThingSpeak exige mínimo de 15 segundos
}

```